R4DS 03 – Data Transformation

Introduction

dplyr: The Data Transformation Toolbox

- **dplyr** provides verbs for the most common data manipulation tasks
- All verbs share a common pattern:
  1. First argument: a data frame
  2. Subsequent arguments: what to do (using column names, no quotes)
  3. Output: a new data frame
- Combine verbs with the **pipe** |>

Prerequisites

```
library(nycflights13)
library(tidyverse)
```

The flights Data

```
flights
```

```
#> # A tibble: 336,776 x 19
#>     year month   day dep_time sched_dep_time dep_delay
#>    <int> <int> <int>    <int>          <int>     <dbl>
#>  1  2013     1     1      517            515         2
#>  2  2013     1     1      533            529         4
#>  3  2013     1     1      542            540         2
#>  4  2013     1     1      544            545        -1
#> ... [output truncated]
```

- 336,776 flights departing NYC in 2013
- Types: <int> integer, <dbl> double, <chr> character, <dttm> date-time

glimpse() — See All Variables

```
glimpse(flights)
```

```
#> Rows: 336,776
#> Columns: 19
#> $ year           <int> 2013, 2013, 2013, 2013, 2013, ~
#> $ month          <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ day            <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ dep_time       <int> 517, 533, 542, 544, 554, 554, ~
#> $ sched_dep_time <int> 515, 529, 540, 545, 600, 558, ~
#> $ dep_delay      <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -~
#> $ arr_time       <int> 830, 850, 923, 1004, 812, 740,~
#> $ sched_arr_time <int> 819, 830, 850, 1022, 837, 728,~
#> $ arr_delay      <dbl> 11, 20, 33, -18, -25, 12, 19, ~
#> $ carrier        <chr> "UA", "UA", "AA", "B6", "DL", ~
#> ... [output truncated]
```

dplyr Verbs at a Glance

Category	Verb	Action
Rows	`filter()`	Keep rows that match conditions
Rows	`arrange()`	Reorder rows
Rows	`distinct()`	Remove duplicate rows
Columns	`mutate()`	Create/modify columns
Columns	`select()`	Pick columns
Columns	`rename()`	Rename columns
Columns	`relocate()`	Reorder columns
Groups	`group_by()`	Define groups
Groups	`summarize()`	Collapse groups to summaries

Rows

`filter()` — Keep Rows by Condition

```
# Flights that departed more than 2 hours late
flights |>
  filter(dep_delay > 120)
```

```
#> # A tibble: 9,723 x 19
#>     year month   day dep_time sched_dep_time dep_delay
#>    <int> <int> <int>    <int>          <int>     <dbl>
#>  1  2013     1     1      848           1835       853
#>  2  2013     1     1      957            733       144
#>  3  2013     1     1     1114            900       134
#> ... [output truncated]
```

`filter()` — Combining Conditions

```
# Flights on January 1
flights |>
  filter(month == 1 & day == 1)
```

```
#> # A tibble: 842 x 19
#>     year month   day dep_time sched_dep_time dep_delay
#>    <int> <int> <int>    <int>          <int>     <dbl>
#>  1  2013     1     1      517            515         2
#>  2  2013     1     1      533            529         4
#> ... [output truncated]
```

```
# Flights in January or February (shortcut with %in%)
flights |>
  filter(month %in% c(1, 2))
```

```
#> # A tibble: 51,955 x 19
#>     year month   day dep_time sched_dep_time dep_delay
#>    <int> <int> <int>    <int>          <int>     <dbl>
#>  1  2013     1     1      517            515         2
#>  2  2013     1     1      533            529         4
#> ... [output truncated]
```

`filter()` — Common Mistakes

Use `==` not `=` for comparison:

```
flights |>
  filter(month = 1)
```

```
#> Error in `filter()`:
#> ! We detected a named input.
#> i This usually means that you've used `=` instead of
#>   `==`.
#> i Did you mean `month == 1`?
```

Use `%in%` or `|`, not English "or":

```
# WRONG: month == 1 | 2  (always TRUE!)
# RIGHT:
flights |> filter(month == 1 | month == 2)
flights |> filter(month %in% c(1, 2))
```

arrange() — Reorder Rows

```
flights |>
  arrange(year, month, day, dep_time)
```

```
#> # A tibble: 336,776 x 19
#>     year month   day dep_time sched_dep_time dep_delay
#>    <int> <int> <int>    <int>          <int>     <dbl>
#>  1  2013     1     1      517            515         2
#>  2  2013     1     1      533            529         4
#> ... [output truncated]
```

Use `desc()` for descending order:

```
flights |>
  arrange(desc(dep_delay))
```

```
#> # A tibble: 336,776 x 19
#>     year month   day dep_time sched_dep_time dep_delay
#>    <int> <int> <int>    <int>          <int>     <dbl>
#>  1  2013     1     9      641            900      1301
#>  2  2013     6    15     1432           1935      1137
#> ... [output truncated]
```

`distinct()` — Unique Rows

```
flights |>
  distinct(origin, dest)
```

```
#> # A tibble: 224 x 2
#>    origin dest
#>    <chr>  <chr>
#>  1 EWR    IAH
#>  2 LGA    IAH
#> ... [output truncated]
```

Use `count()` to also get frequencies:

```
flights |>
  count(origin, dest, sort = TRUE)
```

```
#> # A tibble: 224 x 3
#>    origin dest      n
#>    <chr>  <chr> <int>
#>  1 JFK    LAX   11262
#>  2 LGA    ATL   10263
#> ... [output truncated]
```

Exercises 3.2.5

1. Find all flights that:
   - Arrived $>$ 2 hours late
   - Flew to Houston (`IAH` or `HOU`)
   - Operated by United, American, or Delta
   - Departed in summer (Jul–Sep)
   - Arrived $>$ 2 hours late but didn't leave late
   - Delayed $\geq$ 1 hour but made up $>$ 30 min in flight

Exercises 3.2.5 (cont.)

2. Sort `flights` to find longest departure delays. Find earliest morning flights.
3. Sort to find fastest flights. (Hint: use math in the function.)
4. Was there a flight on every day of 2013?
5. Which flights traveled farthest? Least distance?
6. Does order of `filter()` and `arrange()` matter? Why/why not?

Solutions 3.2.5 (1)

```
# a) Arrived > 2 hours late
flights |> filter(arr_delay > 120)
# b) Flew to Houston
flights |> filter(dest %in% c("IAH", "HOU"))
# c) United, American, or Delta
flights |> filter(carrier %in% c("UA", "AA", "DL"))
# d) Summer
flights |> filter(month %in% c(7, 8, 9))
# e) Arrived >2h late, didn't leave late
flights |> filter(arr_delay > 120 & dep_delay <= 0)
# f) Delayed >=1h, made up >30 min
flights |> filter(dep_delay >= 60 &
                  dep_delay - arr_delay > 30)
```

Solutions 3.2.5 (2–6)

2.

```
flights |> arrange(desc(dep_delay))   # longest delay
flights |> arrange(dep_time)          # earliest morning
```

3. `flights |> arrange(air_time)` or `flights |> arrange(distance / air_time)`
4. `flights |> distinct(month, day) |> nrow()` — yes, 365 rows.
5. `flights |> arrange(desc(distance))` (farthest) and `flights |> arrange(distance)` (shortest).
6. **No** — same rows and same order either way. But filtering first is more efficient (fewer rows to sort).

Columns

`mutate()` — Create New Columns

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60,
    .before = 1
  )
```

```
#> # A tibble: 336,776 x 21
#>     gain speed  year month   day dep_time
#>    <dbl> <dbl> <int> <int> <int>    <int>
#>  1    -9  370.  2013     1     1      517
#>  2   -16  374.  2013     1     1      533
#>  3   -31  408.  2013     1     1      542
#> ... [output truncated]
```

`.before = 1` places new columns at the front.

mutate() — .keep Argument

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    hours = air_time / 60,
    gain_per_hour = gain / hours,
    .keep = "used"
  )
```

```
#> # A tibble: 336,776 x 6
#>    dep_delay arr_delay air_time  gain hours
#>        <dbl>     <dbl>    <dbl> <dbl> <dbl>
#>  1         2        11      227    -9 3.78
#>  2         4        20      227   -16 3.78
#>  3         2        33      160   -31 2.67
#> ... [output truncated]
```

`.keep = "used"` keeps only input and output columns.

`select()` — Pick Columns

```
flights |> select(year, month, day)       # by name
flights |> select(year:day)               # range
flights |> select(!year:day)              # exclude range
flights |> select(where(is.character))    # by type
```

Helper functions: `starts_with()`, `ends_with()`, `contains()`, `num_range()`

Rename while selecting:

```
flights |>
  select(tail_num = tailnum)
```

```
#> # A tibble: 336,776 x 1
#>    tail_num
#>    <chr>
#>  1 N14228
#>  2 N24211
#> ... [output truncated]
```

rename() and relocate()

`rename()` keeps all columns, just renames:

```
flights |> rename(tail_num = tailnum)
```

`relocate()` moves columns:

```
flights |>
  relocate(time_hour, air_time)
```

```
#> # A tibble: 336,776 x 19
#>    time_hour           air_time  year month   day
#>    <dttm>                 <dbl> <int> <int> <int>
#>  1 2013-01-01 05:00:00      227  2013     1     1
#>  2 2013-01-01 05:00:00      227  2013     1     1
#> ... [output truncated]
```

Use `.before` / `.after` to control position.

Exercises 3.3.5

1. Compare `dep_time`, `sched_dep_time`, `dep_delay`. How are they related?
2. Brainstorm ways to select `dep_time`, `dep_delay`, `arr_time`, `arr_delay`.
3. What if you specify the same variable multiple times in `select()`?
4. What does `any_of()` do?
5. Does `select(contains("TIME"))` surprise you?
6. Rename `air_time` to `air_time_min` and move to front.
7. Why doesn't this work?

```
flights |> select(tailnum) |> arrange(arr_delay)
```

Solutions 3.3.5 (1–4)

1. `dep_time = sched_dep_time + dep_delay` (in minutes, though `dep_time` is HHMM format).
2. Several ways:

```
flights |> select(dep_time, dep_delay, arr_time, arr_delay)
flights |> select(starts_with("dep"), starts_with("arr"))
flights |> select(matches("^(dep|arr)_(time|delay)$"))
```

3. Column appears only **once** — duplicates are silently ignored.
4. `any_of()` selects columns from a character vector, ignoring names that don't exist (no error).

Solutions 3.3.5 (5–7)

5. `contains()` is **case-insensitive** by default. Change with `contains("TIME", ignore.case = FALSE)`.
6.

```
flights |>
  rename(air_time_min = air_time) |>
  relocate(air_time_min)
```

7. After `select(tailnum)`, only `tailnum` remains — `arr_delay` no longer exists.

The Pipe

Chaining Verbs with |>

```
# Fastest flights to Houston
flights |>
  filter(dest == "IAH") |>
  mutate(speed = distance / air_time * 60) |>
  select(year:day, dep_time, carrier, flight, speed) |>
  arrange(desc(speed))
```

```
#> # A tibble: 7,198 x 7
#>     year month   day dep_time carrier flight speed
#>    <int> <int> <int>    <int> <chr>    <int> <dbl>
#>  1  2013     7     9      707 UA         226  522.
#>  2  2013     8    27     1850 UA        1128  521.
#> ... [output truncated]
```

Read as: "Take flights, **then** filter, **then** mutate, **then** select, **then** arrange."

Without the Pipe

Nesting (hard to read):

```
arrange(select(mutate(filter(flights,
  dest == "IAH"), speed = distance / air_time * 60),
  year:day, dep_time, carrier, flight, speed),
  desc(speed))
```

Intermediate objects (clutters environment):

```
flights1 <- filter(flights, dest == "IAH")
flights2 <- mutate(flights1, speed = distance / air_time * 60)
flights3 <- select(flights2, year:day, dep_time, carrier, flight, speed)
arrange(flights3, desc(speed))
```

The pipe is cleaner. Shortcut: **Ctrl + Shift + M**.

Groups

group_by() + summarize()

```
flights |>
  group_by(month) |>
  summarize(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n = n()
  )
```

```
#> # A tibble: 12 x 3
#>    month avg_delay     n
#>    <int>     <dbl> <int>
#>  1     1      10.0  27004
#>  2     2      10.8  24951
#>  3     3      13.2  28834
#>  4     4      13.9  28330
#>  5     5      13.0  28796
#> ... [output truncated]
```

- `na.rm = TRUE` — ignore missing values
- `n()` — count rows per group

The .by Syntax (dplyr 1.1.0+)

Per-operation grouping — no need for `ungroup()`:

```
flights |>
  summarize(
    delay = mean(dep_delay, na.rm = TRUE),
    n = n(),
    .by = c(origin, dest)
  )
```

```
#> # A tibble: 224 x 4
#>    origin dest  delay     n
#>    <chr>  <chr> <dbl> <int>
#>  1 EWR    IAH   11.8   3973
#>  2 LGA    IAH    9.06  2951
#>  3 JFK    MIA    9.34  3314
#> ... [output truncated]
```

`.by` works with all verbs.

slice_*() Functions

Function	Action
`slice_head(n = 1)`	First row per group
`slice_tail(n = 1)`	Last row per group
`slice_min(x, n = 1)`	Row with smallest x
`slice_max(x, n = 1)`	Row with largest x
`slice_sample(n = 1)`	Random row per group

```
flights |>
  group_by(dest) |>
  slice_max(arr_delay, n = 1) |>
  relocate(dest)
```

```
#> # A tibble: 108 x 19
#> # Groups:   dest [105]
#>    dest   year month   day dep_time sched_dep_time
#>    <chr> <int> <int> <int>    <int>          <int>
#>  1 ABQ    2013     7    22     2145           2007
#> ... [output truncated]
```

Grouping by Multiple Variables

```r
daily <- flights |> group_by(year, month, day)
daily |> summarize(n = n())
```

```
#> # A tibble: 365 x 4
#> # Groups:   year, month [12]
#>     year month   day     n
#>    <int> <int> <int> <int>
#>  1  2013     1     1   842
#>  2  2013     1     2   943
#> ... [output truncated]
```

Each `summarize()` peels off the last group. Use `.groups = "drop"` to drop all.

Exercises 3.5.7 (1–5)

1. Which carrier has the worst average delays?
2. Most delayed departure flight to each destination.
3. How do delays vary over the course of the day? (Plot it.)
4. What happens with negative `n` in `slice_min()`?
5. Explain `count()` in terms of dplyr verbs. What does `sort` do?

Exercise 3.5.7 (6)

Given:

```
df <- tibble(
  x = 1:5,
  y = c("a", "b", "a", "a", "b"),
  z = c("K", "K", "L", "L", "K")
)
```

Predict the output of each, then check:

a. `df |> group_by(y)`
b. `df |> arrange(y)`
c. `df |> group_by(y) |> summarize(mean_x = mean(x))`
d. `df |> group_by(y, z) |> summarize(mean_x = mean(x))`
e. Same as (d) but with `.groups = "drop"`
f. Compare `summarize(mean_x = mean(x))` vs. `mutate(mean_x = mean(x))` after `group_by(y, z)`

Solutions 3.5.7 (1–3)

1.

```
flights |>
  group_by(carrier) |>
  summarize(avg_delay = mean(dep_delay, na.rm = TRUE)) |>
  arrange(desc(avg_delay))
```

```
#> # A tibble: 16 x 2
#>    carrier avg_delay
#>    <chr>       <dbl>
#>  1 F9           20.2
#>  2 EV           20.0
#> ... [output truncated]
```

Hard to disentangle airport vs. carrier effects (confounded).

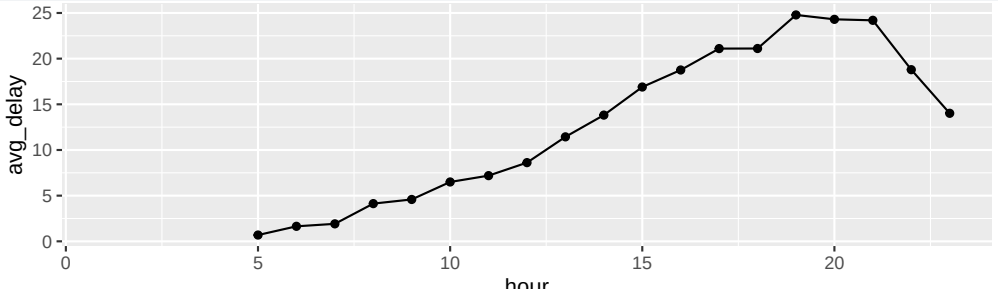
Solutions 3.5.7 (2–3)

❷

```
flights |>
  group_by(dest) |>
  slice_max(dep_delay, n = 1)
```

❸

```
flights |>
  group_by(hour) |>
  summarize(avg_delay = mean(dep_delay, na.rm = TRUE)) |>
  ggplot(aes(x = hour, y = avg_delay)) +
  geom_line() + geom_point()
```



Solutions 3.5.7 (4–6)

4. Negative `n` removes that many rows (keeps all except the n smallest/largest).
5. `count(x)` is equivalent to `group_by(x) |> summarize(n = n())`. The `sort = TRUE` argument arranges results in descending order of `n`.
6. a. Same data, but grouped (header shows `Groups: y [2]`).
   b. Rows reordered by `y` (a's first, then b's). Not grouped.
   c. Two rows: mean of `x` for `y = "a"` (mean 3) and `y = "b"` (mean 3.5).
   d. Three rows (one per y-z combo). Message: "grouped by y".
   e. Same three rows, but result is **ungrouped**.
   f. `summarize` collapses to 3 rows; `mutate` keeps all 5 rows, adding the group mean as a new column.

Case Study

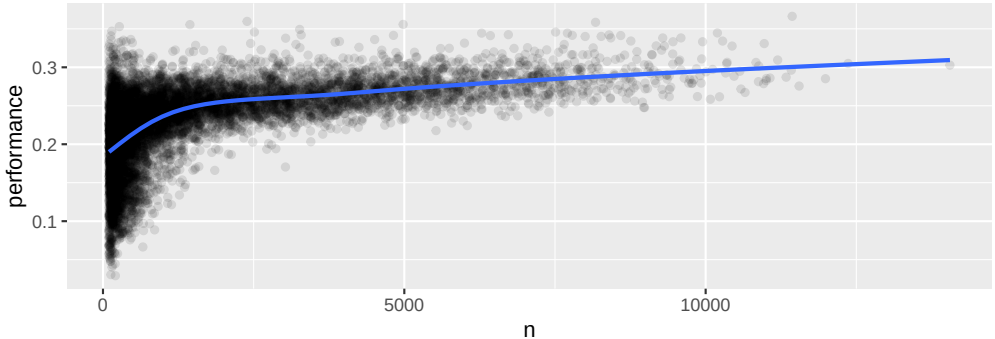
Aggregates and Sample Size

```
batters <- Lahman::Batting |>
  group_by(playerID) |>
  summarize(
    performance = sum(H, na.rm = TRUE) / sum(AB, na.rm = TRUE),
    n = sum(AB, na.rm = TRUE)
  )
batters
```

```
#> # A tibble: 20,985 x 3
#>    playerID  performance     n
#>    <chr>           <dbl> <int>
#>  1 aardsda01       0         4
#>  2 aaronha01       0.305 12364
#> ... [output truncated]
```

Variation Decreases with Sample Size

```
batters |>
  filter(n > 100) |>
  ggplot(aes(x = n, y = performance)) +
  geom_point(alpha = 0.1) +
  geom_smooth(se = FALSE)
```



Always include `n()` when aggregating — don't draw conclusions from tiny groups.